Laboratory Report: Logger and Parentheses Validator

Student: Nikita Moglan, FAF-252
Programming language: Java

1. Program Structure

Logger prints messages. It checks if the level of a message is high enough and builds the line from the format string. ParenthesesValidator checks if a string with parentheses is valid and returns true or false. The validator has its own Logger called "Parentheses" and calls trace, info, warn and error on it, so it uses the logger but does not inherit from it. Main just runs the tests.

2. Classes and Objects

The class is the description and the object is the actual thing in memory. For example new Logger("Parentheses") inside the validator creates one logger object, and its only state is its name. The validator object keeps its logger. The counter open is a local variable in isValid, so it is not really part of the state, it starts from 0 on every call.

3. Static and Instance Members

```
private String name;                                // per object
public static LogLevel level = LogLevel.INFO;      // shared
public static String format = "[%t] [%l] %n: %m";  // shared
```

level and format are static because they should be the same for all loggers, so there is one copy that belongs to the class. name is different for every logger, so it is an instance field, and since log uses name, log cannot be static either. I found this out the hard way: first I made log static and got "Non-static field 'name' cannot be referenced from a static context", then I made name static and all loggers started printing the name of the last one created. When I call Logger.setLevel(WARN) in my test, both the "First" and "Second" loggers stop printing INFO, because they read the same field.

4. Methods and Responsibilities

log compares msgLevel.ordinal() with level.ordinal() and returns if the message is too low. The enum order is TRACE, INFO, WARN, ERROR, so this is just comparing numbers. Then it replaces the placeholders on a copy of format, because changing format itself would break it for the next call. %m goes last so nothing inside the message gets replaced. The validator uses a logger instead of printing by itself so the formatting and filtering stay in one place: if I want to hide trace messages I change the level, not the validator. An example of a call between objects is logger.trace("Opening parenthesis found at position " + i).

5. Testing and Correctness

(())((()))(()) is valid,)((())) is not because it starts with a closing parenthesis. I keep a counter that goes +1 for (and -1 for). If a) comes when the counter is 0, the string is invalid right away, and at the end the counter has to be 0. The 4 given tests pass, and so do (()), ((), ()) and the empty string. The results are the same on TRACE, INFO and ERROR, only the amount of output changes. The test with two loggers is the one that would have caught my bug with the static name.

6. Reflection

Static vs instance was the main thing I learned here, since most of my errors were about it. With more time I would make level and format private, because now they are public to match the UML and anyone can change them. I would also use a stack instead of a counter, so it works with [] and {} too.